CURA • ADAPTIVE AI ASSISTANT

المتطلبات الفنية وخارطة طريق فريق الذكاء الاصطناعي

المطلوب بالكامل من **تيم الـ AI** لتشغيل منظومة **Cura** بعد استبعاد الكاميرا وتوجيه موارد الهاردوير بالكامل للمعالجة الصوتية واللغوية والربط بالإنترنت.

أولاً: تجهيز وتشغيل خط أنابيب النماذج المحلية (On-Device AI Pipeline)

2.2GB – 2.8GB RAM

بدلاً من استهلاك المعالج والرامات في موديول الكاميرا والتعرف على الوجوه (`FR-HW-003`)، يتم التركيز على تشغيل **3 موديلات محلية خفيفة** تعمل معاً كمسار واحد (Pipeline) عبر `Ollama` أو `llama.cpp` بحيث لا يتخطى استهلاك الرامات **2.2GB إلى 2.8GB RAM**:

1. الموديل اللغوي الأساسي (LLM):

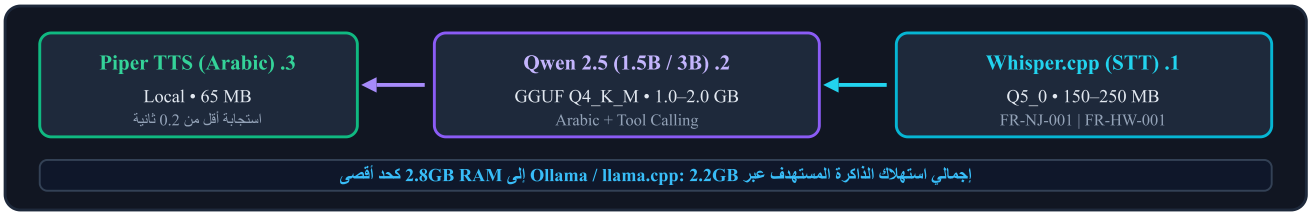
- تحميل وتشغيل موديل **Qwen 2.5 (1.5B-Instruct)** بصيغة مضغوطة `GGUF Q4_K_M` (حجمه حوالي **1.0 GB** فقط) لدعمه القوي للغة العربية وميزة **Tool Calling**، وسرعته العالية على اللاب توب (كارت شاشة 2GB VRAM) وعلى بوردة الـ Raspberry Pi.
- تجهيز البديل الأقوى **Qwen 2.5 (3B-Instruct)** أو **Gemma 2 (2B)** بصيغة `Q4_K_M` (بحجم **1.7 إلى 2.0 GB**) في حال التشغيل على Raspberry Pi 5 أو اللاب توب.

2. موديل تحويل الصوت إلى نص (STT - Speech-to-Text):

- إعداد **Whisper.cpp** (نسخة `base` أو `small` بصيغة `Q5_0` بحجم **150 إلى 250 MB**) لالتقاط كلام المريض أو أهله بالعربية وتحويله لنص محلياً بدقة (`FR-NJ-001` و `FR-HW-001`).

3. موديل نطق الكلام بالصوت (TTS - Text-to-Speech):

- إعداد **Piper TTS** (الموديل العربي المحمل محلياً بحجم **65 MB**) لنطق التنبيهات والردود مثل *"خد الدوا يا أستاذ حسين"* أو *"خلص التاسك يا أحمد"* في أقل من **0.2 ثانية** بدون استهلاك يُذكر من المعالج.



ثانياً: بناء منطق الأنماط الثلاثة وهندسة الأوامر (Adaptive Modes & System Prompts)

3 Adaptive Modes

1. منطق وضع الزهايمر (Alzheimer's Care Logic)

`FR-ALZ-001 | 007 | 009`

- **تأكيد الجرعات بالصوت (`FR-ALZ-007`):** تحليل رد المريض الصوتي بعد سماعه لتنبيه الدواء (مثل: *"أخدته خلاص"*)، واستخراج نية التأكيد (Intent Detection) لإرسال إشارة للسيرفر بتسجيل الجرعة في الداتابيز.
- **رصد الأسئلة المتكررة والارتباك (`FR-ALZ-009`):** مقارنة أسئلة المريض بسجل المحادثات الأخير لرصد تكرار نفس السؤال وإرسال إحصائية التكرار للـ API، مع الرد على المريض بنبرة مطمئنة وهادئة في كل مرة.
- **تقرير حالة المريض الصوتي للأهل (`FR-ALZ-001`):** عندما يسأل أحد أهل المريض *"عمل ايه آخر يومين؟"*، يقوم كود البايثون بسحب بيانات آخر يومين من الـ API (مرات أخذ الدواء، الأسئلة المتكررة، المهام) ووضعها في Prompt قصير لموديل Qwen 2.5 لتلخيصها في جملتين بالعامية المصرية ونطقها فوراً.

2. منطق وضع فرط الحركة وتشتت الانتباه (ADHD Focus Logic)

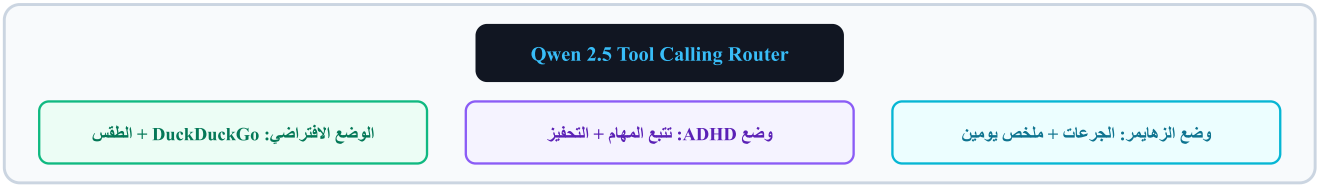
`FR-ADHD-003 | 010 | 011`

- **متابعة وتأكيد المهام (`FR-ADHD-003` و `FR-ADHD-010`):** استقبال تنبيهات المهام ونطقها لتحفيز المستخدم، ثم فهم رده الصوتي عند إنهاء التاسك لتحديث حالته في قاعدة البيانات وتلخيص الإنجاز (`FR-ADHD-011`).
- **التأني والتشجيع الصوتي:** توليد رسائل تحفيزية قصيرة وتذكيرات بالتنفس والتركيز على المهمة الواحدة.

3. منطق الوضع الافتراضي والبحث بالإنترنت (Normal Mode & Web Search)

`FR-NJ-001 | 005`

- **البحث اللحظي والطقس (`FR-NJ-005`):** ربط الموديل بالإنترنت عبر مكتبة `duckduckgo-search` و `OpenWeatherMap API` بحيث عندما يسأل المستخدم *"درجة الحرارة كام؟"* أو عن معلومة عامة، يجلب سكريبت البايثون النتيجة ويمررها لموديل Qwen 2.5 لصياغتها والرد بها صوتياً (`FR-NJ-001`).



ثالثاً: بناء طبقة الربط والتنسيق البرمجي (Python Orchestrator & API Bridge)

FastAPI / WebSockets

تطوير سكريبت بايثون مركزي (وخدمة FastAPI/WebSockets محلية) يعمل على البوردة والـ Desktop GUI ليقوم بالمهام التالية:

1. إدارة حلقة التفاعل الصوتي (Voice Interaction Loop):

- التقاط الصوت من المصفوفة (Microphone Array) ← تمريره لـ **Whisper.cpp** ← إرسال النص لموديل **Qwen 2.5** ← تحويل الرد لصوت عبر **Piper TTS** وإخراجه من السماعة (Speaker).

2. التحكم في حالات الهاردوير والشاشة (Hardware & Screen State Sync):

- تغيير حالة إضاءة الـ **RGB LED Ring** (`FR-HW-006`) بالتزامن مع مراحل عمل الـ AI: أزرق عند الاستماع (Listening)، أخضر أثناء المعالجة (Processing)، أصفر أثناء النطق (Speaking)، وأحمر في الطوارئ (Alert/Emergency).
- إرسال النصوص والحالات اللحظية للشاشة الـ 4 بوصة (`FR-HW-007`) ولواجهة الـ Desktop GUI.

3. مستقبل الإشعارات اللحظية (Real-Time Event Listener):

- فتح اتصال دائم (عبر WebSockets أو Firebase أو MQTT) مع السيرفر لاستقبال مواعيد الأدوية والمهام اللحظية (`FR-ALZ-006` و `FR-ADHD-003`) وتشغيل التنبيه الصوتي والمرئي تلقائياً دون انتظار تدخل من المريض.

حالات إضاءة RGB LED Ring (FR-HW-006):	أزرق: الاستماع (Listening)	أخضر: المعالجة (Processing)	أصفر: النطق (Speaking)	أحمر: الطوارئ (Alert/Emergency)

رابعاً: جدول تسليمات تيم الـ AI بالترتيب (Actionable Roadmap)

4 Milestones

المرحلة	التاسك المطلوب	المخرجات النهائية (Deliverables)
المرحلة 1 (على اللاب توب)	**إعداد البيئة المحلية وتشغيل الـ 3 موديلات**	تشغيل Ollama + Qwen 2.5 (1.5B) + Whisper.cpp + Piper TTS في سكريبت Python واحد يأخذ صوتاً بالعربية ويرد بصوت عربي سريع على كارت الـ 2GB VRAM.
المرحلة 2 (Tool Calling)	**برمجة الأدوات والربط بالإنترنت والـ API**	كتابة دوال البايثون الخاصة بالبحث (duckduckgo-search)، الطقس (OpenWeatherMap)، دالة تلخيص حالة المريض لآخر يومين، ودالة تأكيد أخذ الدواء وإنهاء التاسك.
المرحلة 3 (AI Local Server)	**تغليف الـ AI في خدمة محلية للـ GUI والهاردوير**	بناء API / WebSocket محلي يربط سكريبت الـ AI بشاشة الـ GUI ولمبات الـ LED الفيزيائية لتغيير ألوانها (أزرق/أخضر/أصفر/أحمر) حسب حالة الموديل.
المرحلة 4 (النقل على الـ Pi)	**النشر واختبار الأداء على الـ Raspberry Pi**	نقل المنظومة على كارت الميموري السريع (SanDisk Extreme A2) على الـ Raspberry Pi واختبار سرعة الرد الصوتي واستقرار استهلاك الرامات مع اتصال الـ Wi-Fi (FR-HW-008).